

Regular Program Solutions

Insertion Sort - Part 1

```
#include <stdio.h>
void print(int ar_size, int* ar) {
    int i;
    for(i=0; i<ar_size; i++) {
        printf("%d ", ar[i]);
    }
    printf("\n");
}

#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <assert.h>
/* Head ends here */
void insertionSort(int ar_size, int * ar) {
    int j = ar_size-1;
    int v = ar[j];
    while(v < ar[j-1]) {
        ar[j] = ar[j-1];
        j--;
    }
    print(ar_size, ar);
    ar[j] = v;
    print(ar_size, ar);
}
/* Tail starts here */
int main() {
    int _ar_size;
    scanf("%d", &_ar_size);
    int _ar[_ar_size], _ar_i;
    for(_ar_i = 0; _ar_i < _ar_size; _ar_i++) {
        scanf("%d", &_ar[_ar_i]);
    }
    insertionSort(_ar_size, _ar);
    return 0;
}
```

Insertion Sort - Part 2

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <assert.h>
/* Head ends here */
void insertionSort(int ar_size, int * ar) {
    for (int i = 1; i < ar_size; ++i) {
        int j = i - 1;
        int p = ar[i];
        while (j >= 0 && p < ar[j]) {
            ar[j+1] = ar[j];
            j--;
        }
        ar[j+1] = p;
        printf("%d", ar[0]);
        for (int k = 1; k < ar_size; ++k) {
            printf(" %d", ar[k]);
        }
        printf("\n");
    }
}
/* Tail starts here */
int main() {
    int _ar_size;
    scanf("%d", &_ar_size);
    int _ar[_ar_size], _ar_i;
    for(_ar_i = 0; _ar_i < _ar_size; _ar_i++) {
        scanf("%d", &_ar[_ar_i]);
    }
    insertionSort(_ar_size, _ar);
    return 0;
}
```

Compare the triplets:

```
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>
int main(){
    int a0;
    int a1;
    int a2;
    scanf("%d %d %d",&a0,&a1,&a2);
    int b0;
    int b1;
    int b2;
    int a_score = 0;
    int b_score = 0;
    scanf("%d %d %d",&b0,&b1,&b2);
    if (a0 > b0)
        a_score++;
    else if (a0 < b0)
        b_score++;
    else{}
    //no op
    if (a1 > b1)
        a_score++;
    else if (a1 < b1)
        b_score++;
    else {}
    //no op
    if (a2 > b2)
        a_score++;
    else if (a2 < b2)
        b_score++;
    else {}
    //no op
    printf("%d %d",a_score, b_score);
    return 0;
}
```

Diagonal Difference

```
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>
int main()
{
    int n, j;
    int i=0, RightDiagonalSum=0, LeftDiagonalSum=0, firstarray, secondarray;
    scanf("%d",&n);
    int a[n][n];
    for( firstarray = 0; firstarray < n; firstarray++)
    {
        for( secondarray = 0; secondarray < n; secondarray++)
        {
            scanf("%d",&a[firstarray][secondarray]);
        }
    }
    while(i<n)
    {
        RightDiagonalSum=RightDiagonalSum+a[i][i];
        i++;
    }
    j=n-1,i=0;
    while(i<n)
    {
        LeftDiagonalSum=LeftDiagonalSum+a[i][j];
        i++;
        j--;
    }
    printf("%d",abs(RightDiagonalSum-LeftDiagonalSum));
    return 0;
}
```

Counting Sort 1

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
int main() {
    int n,i;
    int b[100],a;
    scanf("%d",&n);
    for(i=0;i<100;i++)
    {
        b[i]=0;
    }
    for(i=0;i<n;i++)
    {
        scanf("%d",&a);
        b[a]++;
    }
    for(i=0;i<100;i++)
    {
        printf("%d ", b[i]);
    }
    return 0;
}
```

Non-Divisible Subset

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
// Helper functions for min and max
int min(int a, int b) {
    return a < b ? a : b;
}
int max(int a, int b) {
    return a > b ? a : b;
}
int main() {
    int n, k, a, total = 0;
    // Read input n and k
    scanf("%d %d", &n, &k);
    // Array to store counts of each remainder from 0 to k-1
    int mods[k];
    for (int i = 0; i < k; i++) {
        mods[i] = 0;
    }
    // Read all numbers and count their remainders
    for (int i = 0; i < n; i++) {
        scanf("%d", &a);
        mods[a % k]++;
    }
    // Add at most one element with remainder 0
    total += min(1, mods[0]);
    // Handle complementary remainders
    for (int d = 1; d < (k + 1) / 2; d++) {
        // Pick the maximum count between remainder d and remainder k-d
        total += max(mods[d], mods[k - d]);
    }
    // If k is even, add at most one element with remainder k/2
    if (k % 2 == 0) {
        total += min(1, mods[k / 2]);
    }
    // Print the size of the largest non-divisible subset
    printf("%d\n", total);
    return 0;
}
```

Counter Game

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
int isPow2(long unsigned int);
unsigned long int largePow(long unsigned int);
int main() {
    int t,i,win;
    long unsigned int n;
    scanf("%d",&t);
    for(i=0;i<t;++i)
    {
        win=0;
        scanf("%lu",&n);
        if(n==1)
            printf("Richard\n");
        else
        {
            while(n!=1)
            {
                if(isPow2(n))
                    n>>=1;
                else
                    n-=largePow(n);
                ++win;
            }
            if(win%2==0)
                printf("Richard\n");
            else
                printf("Louise\n");
        }
        return 0;
    }
    int isPow2(long unsigned int n)
    {
        return !(n&(n-1));
    }
    long unsigned int largePow(long unsigned int n)
    {
        long unsigned int m;
        while(n)
        {
            m=n;
            n=n&(n-1);
        }
        return m;
    }
}
```

Sherlock and Cost

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
int main() {
    int T,N,B,L,R,ML,MR,X,Y,P,Q;
    scanf("%d",&T);
    for(int i = 0; i < T; i++) {
        scanf("%d",&N);
        for(int j = 0; j < N; j++) {
            scanf("%d",&B);
            if(j) {
                X = L - 1 + ML;
                Y = R - 1 + MR;
                P = abs(L - B) + ML;
                Q = abs(R - B) + MR;
                ML = (X > Y ? X : Y);
                MR = (P > Q ? P : Q);
            } else {
                ML = MR = 0;
            }
            L = 1;
            R = B;
        }
        printf("%d\n", (ML > MR ? ML : MR));
    }
    return 0;
}
```


SEARCHING AND SORTING

Closest Numbers

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void quickSort(int[], int, int);
```

```
int partition(int[], int, int);
```

```
int main(){
```

```
    int i,n;
```

```
    //input
```

```
    scanf("%d",&n);
```

```
    int *a=(int *)malloc(n*sizeof(int));
```

```
    for(i=0;i<n;i++)
```

```
        scanf("%d",&a[i]);
```

```
    //sorting
```

```
    quickSort(a,0,n-1);
```

```
    //finding smallest
```

```
    int min=a[1]-a[0];
```

```
    for(i=2;i<n;i++)
```

```
        if(a[i]-a[i-1]<min) min=a[i]-a[i-1];
```

```
    //printing all pairs
```

```
    for(i=1;i<n;i++)
```

```
        if(a[i]-a[i-1]==min) printf("%d %d ",a[i-1],a[i]);
```

```
printf("\n");
```

```
return 0;
```

```
}
```

```
void quickSort(int a[], int l, int r)
```

```
{
```

```
    int j;
```

```
    if( l < r )
```

```
    {
```

```
        // divide and conquer
```

```
        j = partition( a, l, r);
```

```
        quickSort( a, l, j-1);
```

```
        quickSort( a, j+1, r);
```

```
    }
```

```
}
```

```
int partition(int a[], int l, int r) {
```

```
    int pivot, i, j, t;
```

```
    pivot = a[l];
```

```
    i = l; j = r+1;
```

```
    while( 1)
```

```
    {
```

```
        do ++i; while( a[i] <= pivot && i <= r );
```

```
        do --j; while( a[j] > pivot );
```

```
        /*do ++i; while( a[i] >= pivot && i <= r );
```

```
        do --j; while( a[j] < pivot );*/
```

```
        if( i >= j ) break;
```

```
        t = a[i]; a[i] = a[j]; a[j] = t;
```

```

    }
    t = a[l]; a[l] = a[j]; a[j] = t;
    return j;
}

```

Ice Cream Parlor

```

#include<stdio.h>

int main()
{
    int t,c,l,i,j,arr[20000];
    scanf("%d",&t);
    for( ; t>0 ; t--)
    {
        scanf("%d%d",&c,&l);
        for(i=0;i<l;i++)
            scanf("%d",&arr[i]);
        for(i=0;i<l-1;i++)
            for(j=i+1;j<l;j++)
            {
                if(arr[i]+arr[j]==c)
                    printf("%d %d\n",i+1,j+1);
            }
    }
    return 0;
}

```

Find the Median

RECURSION AND BIT MANIPULATION

Maximizing XOR

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <assert.h>
int maxXor(int l, int r) {
    int max = 0,i,j;
    for(i=l;i<r;i++)
        for(j=i+1;j<=r;j++)
            max = max<(i^j)?i^j:max;
    return max;
}
int main() {
    int res;
    int _l;
    int _r;
    scanf("%d", &_l);
    scanf("%d", &_r);
    res = maxXor(_l, _r);
    printf("%d", res);
    return 0;
}
```

Sum vs XOR

```
#include <stdio.h>

int main(){
    long long int n,m=1;
    scanf("%lld",&n);
    while(n>0){
        if(n%2==0)m*=2;
        n/=2;
    }
    printf("%lld\n",m);
    return 0; }
```

Flipping Bits

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>

int main() {
    int t;
    unsigned int n;
    scanf("%d", &t);
    while(t-- > 0) {
        scanf("%u", &n);
        printf("%u\n", ~n);
    }
    return 0;
}
```

GREEDY AND DYNAMIC PROGRAMMING

Mark and Toys

```
#include<stdio.h>

void quicksort(int x[100000],int first,int last){
    int pivot,j,temp,i;

    if(first<last){
        pivot=first;
        i=first;
        j=last;

        while(i<j){
            while(x[i]<=x[pivot]&& i<last)
                i++;
            while(x[j]>x[pivot])
                j--;
            if(i<j){
                temp=x[i];
                x[i]=x[j];
                x[j]=temp;
            }
        }

        temp=x[pivot];
        x[pivot]=x[j];
```

```
    x[j]=temp;
    quicksort(x,first,j-1);
    quicksort(x,j+1,last);

}}
```

```
int main()
{
    int n,k,i,avail=0,count=0;
    int cost[n];
    scanf("%d",&n);
    scanf("%d",&k);
    for(i=0;i<n;i++)
        scanf("%d",&cost[i]);
    quicksort(cost,0,n-1);
    while(avail<=k)
    {
        avail+=cost[count];
        count++;
    }
    printf("%d\n",count-1);
    return 0; }
```